

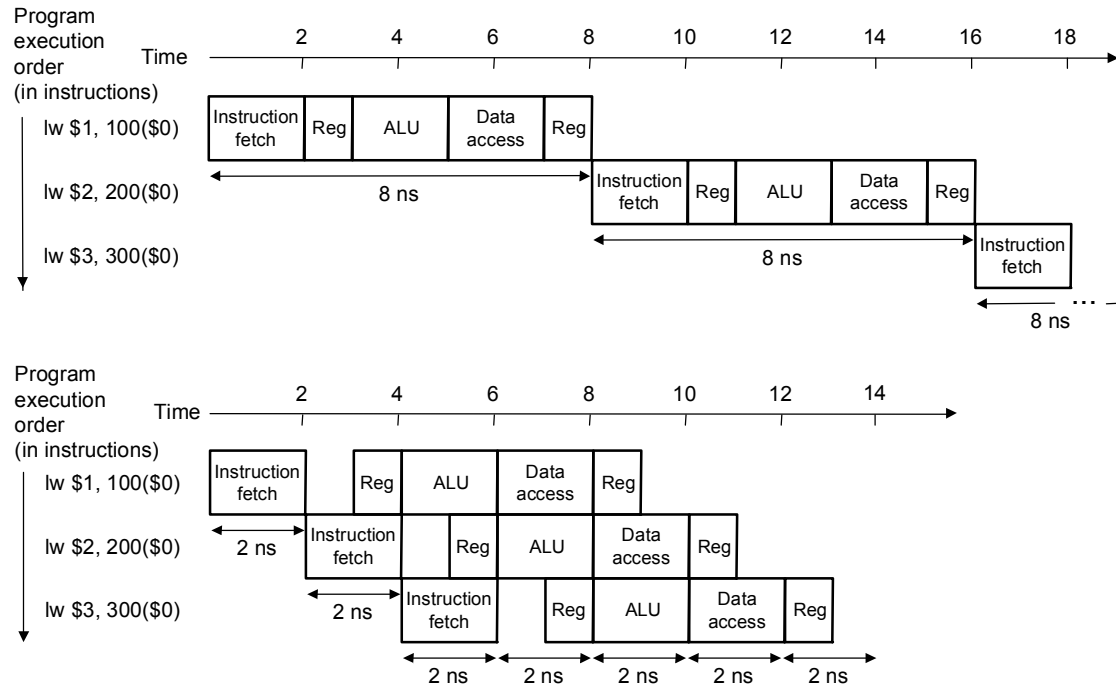
---

# Pipeline

**Capítulo 6 (Patterson e Hennessy, Organização e Projeto de Computadores – 3ª edição)**

# Pipeline

- Melhorar o desempenho aumentando o throughput (instruções por ciclo)



*Speedup ideal é o número de estágios do pipeline. Isto é possível?*

# Desempenho

---

- **Controle Monociclo**
  - Ciclo limitado pela instrução de maior tempo de execução (load)
  - $CPI = 1$
  - No exemplo:  $T_{\text{ciclo}} = 8\text{ns}$
- **Controle Multiciclo**
  - Ciclo limitado pela unidade com maior tempo de execução (memória)
  - $CPI_{\text{min}} = 3$  e  $CPI_{\text{max}} = 5$
  - No exemplo:  $T_{\text{ciclo}} = 2\text{ns}$
- **Pipeline**
  - Ciclo limitado pela unidade com maior tempo de execução (memória)
  - $CPI_{\text{min}} \rightarrow 1$  (infinitas instruções sem conflito) e  $CPI_{\text{max}} = 5$  (uma única instrução)
  - No exemplo:  $T_{\text{ciclo}} = 2\text{ns}$

# Desempenho

---

- **Custo do Pipeline**
  - Overhead da movimentação de dados entre estágios do pipeline
  - Lógica de controle mais complexas (dependências entre memória e registradores)
- **Seja**
  - $t$  = atraso máximo por estágio
  - $k$  = número máximo de estágio
  - $d$  = atraso máximo para propagação de sinais entre estágios (irrelevante)
  - $n$  = número de instruções
  - $T_{k,n}$  = tempo para executar  $n$  instruções em  $k$  estágios

$$Speedup_{k,n} = \frac{T_{1,n}}{T_{k,n}} = \frac{n * k * t}{[k + (n - 1)] * t} = \frac{n * k}{k + (n - 1)}$$

# Pipeline no MIPS (Microprocessor without Interlocked Pipeline Stages)

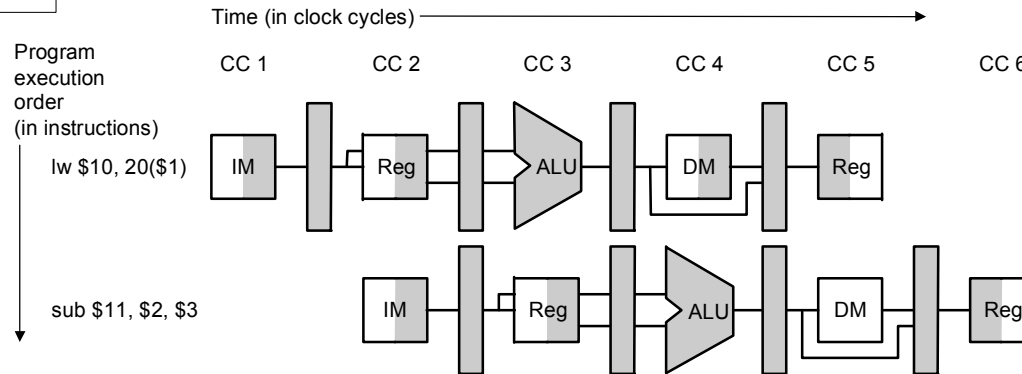
---

- **Por que é fácil?**
  - Instruções possuem o mesmo tamanho
  - Apenas alguns tipos de formato de instruções
  - Operandos de memória aparecem apenas em loads and stores
- **Por que é difícil?**
  - Conflitos estruturais: se fosse o caso de apenas uma memória
  - Conflito de controle: próxima instrução seguindo um desvio
  - Conflito de dados: uma instrução depende do resultado de uma instrução anterior
- **MIPS (without interlocked)**
  - Período de cada estágio constante
  - Estágios não precisam confirmar que o estágio seguinte está livre
  - Memória de dados e Memória de Instruções separadas (evita conflito estrutural)

# Representando Pipelines Graficamente

\*IM = Memória de Instruções

\*DM = Memória de Dados



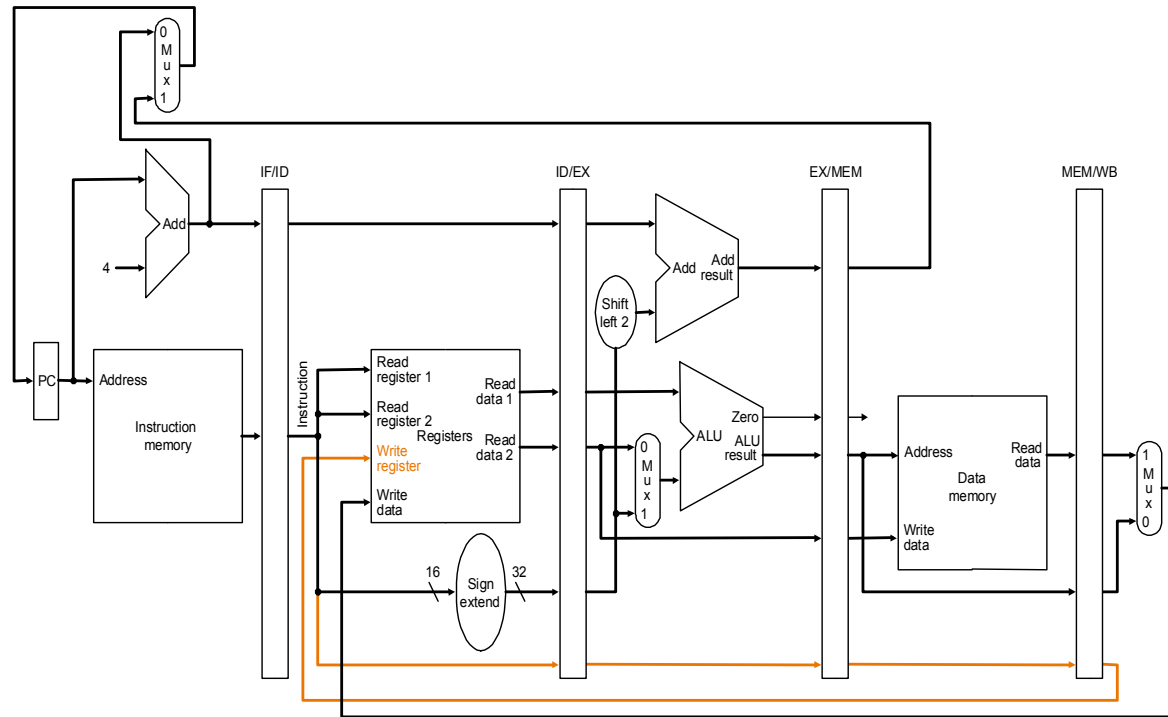
- **Ajuda a responder perguntas como:**
  - Quantos ciclos são necessários para executar um determinado código?
  - O que a ULA está fazendo durante o ciclo 4?
  - Onde pode ocorrer conflitos entre instruções?

# Controle do Pipeline

---

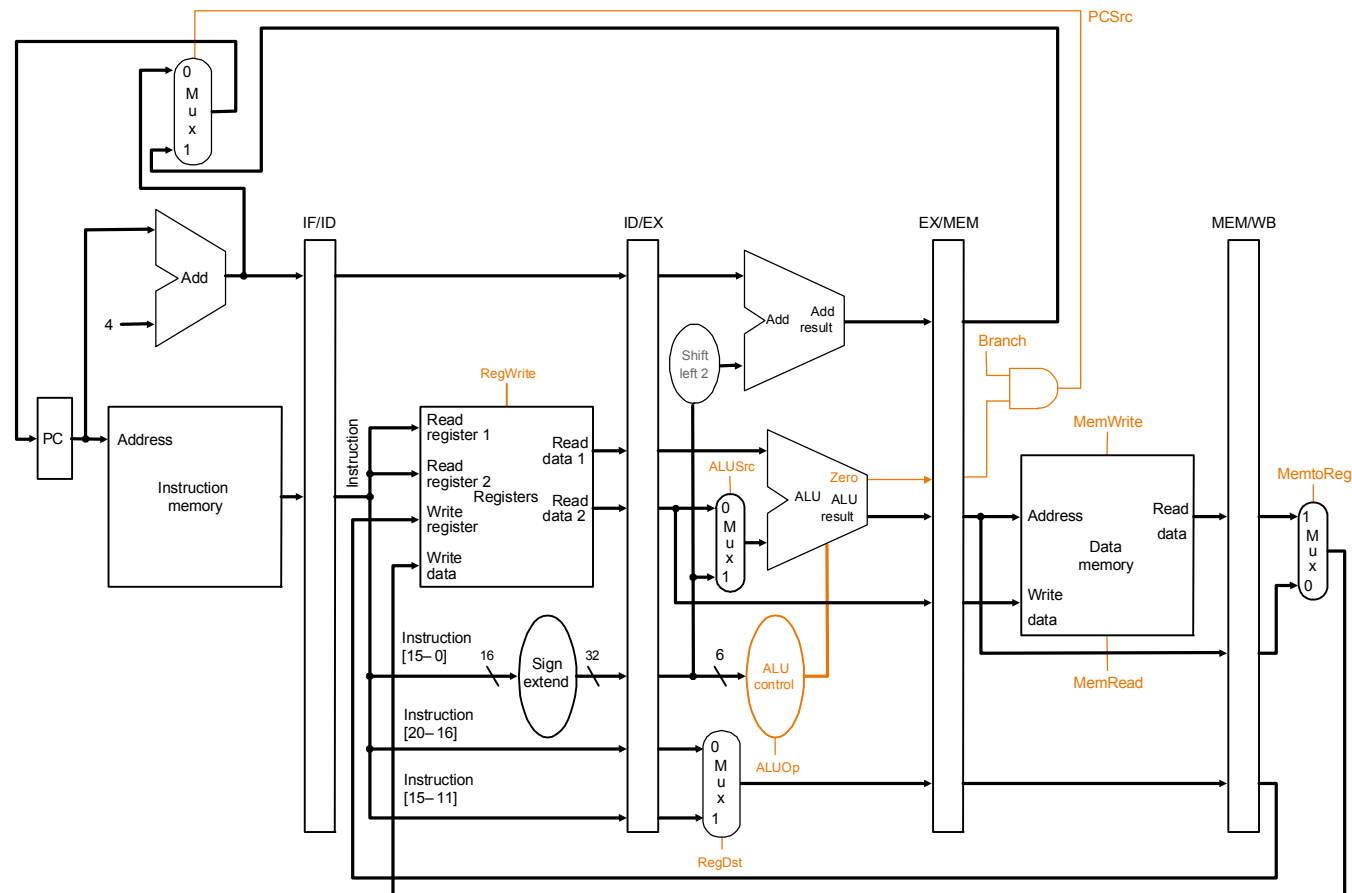
- **5 estágios**
  - **Busca de Instruções (Instruction Fetch – IF)**
  - **Decodificação de Instrução (Instruction Decode - ID)**
  - **Execução (EX)**
  - **Acesso a Memória (MEM)**
  - **Escreve no Registrador (Write Back - WB)**
- **O que é necessário para controlar cada estágio?**
  - **Máquina de estados finita?**
  - **Registradores especiais?**

# Caminho de Dados com Pipeline





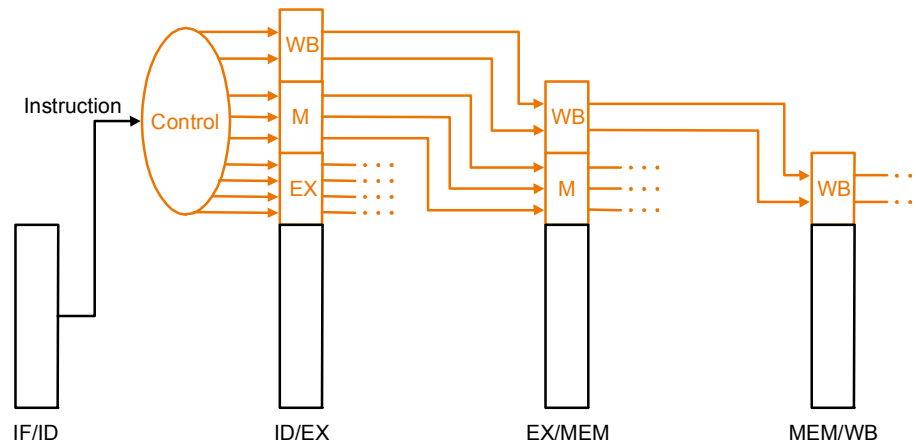
# Caminho de Dados com Pipeline



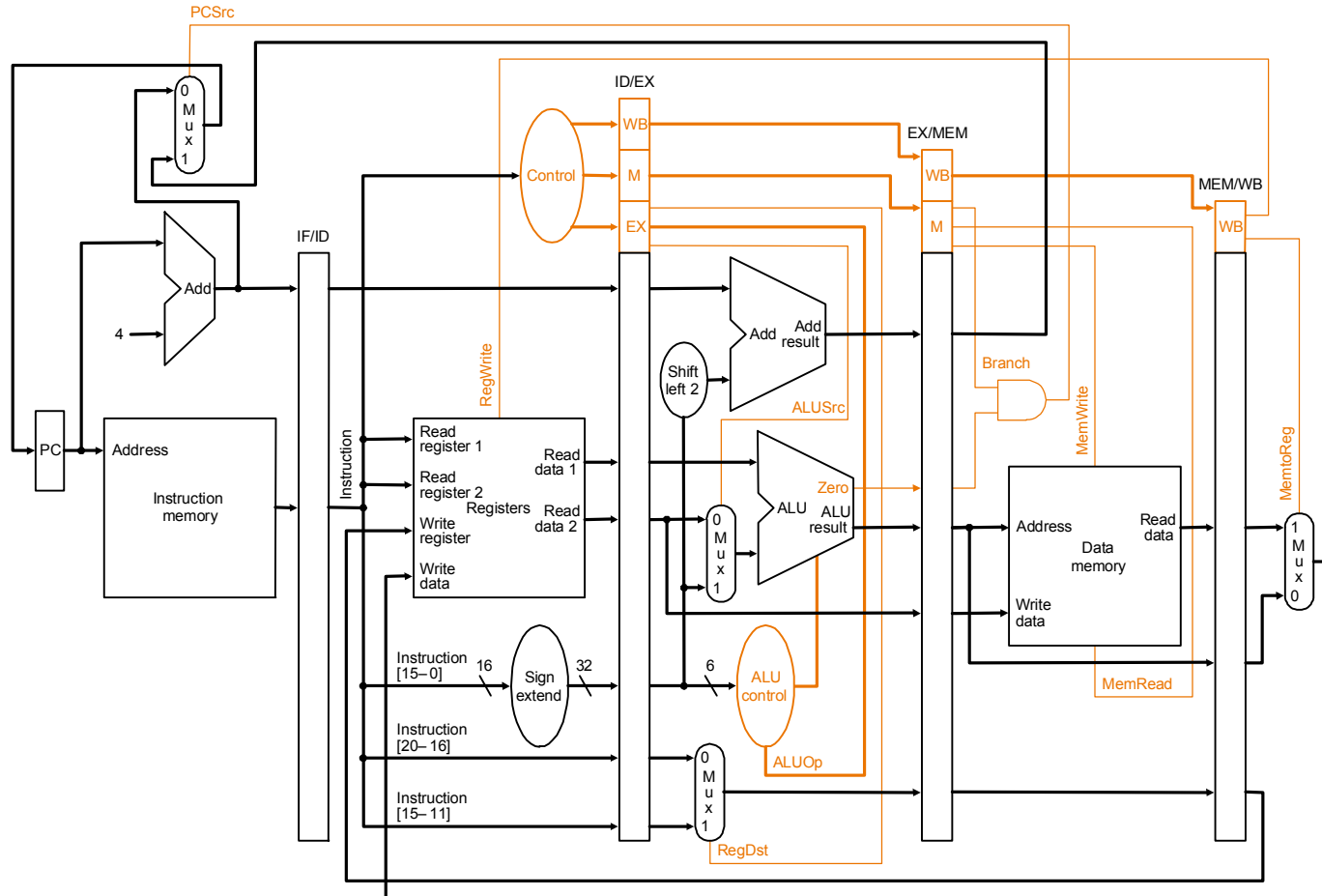
# Caminho de Dados com Pipeline

- Passa sinais de controle para frente

| Instruction | Execution/Address Calculation<br>stage control lines |            |            |            | Memory access stage<br>control lines |             |              | stage control<br>lines |               |
|-------------|------------------------------------------------------|------------|------------|------------|--------------------------------------|-------------|--------------|------------------------|---------------|
|             | Reg<br>Dst                                           | ALU<br>Op1 | ALU<br>Op0 | ALU<br>Src | Branch                               | Mem<br>Read | Mem<br>Write | Reg<br>write           | Mem to<br>Reg |
| R-format    | 1                                                    | 1          | 0          | 0          | 0                                    | 0           | 0            | 1                      | 0             |
| lw          | 0                                                    | 0          | 0          | 1          | 0                                    | 1           | 0            | 1                      | 1             |
| sw          | X                                                    | 0          | 0          | 1          | 0                                    | 0           | 1            | 0                      | X             |
| beq         | X                                                    | 0          | 1          | 0          | 1                                    | 0           | 0            | 0                      | X             |

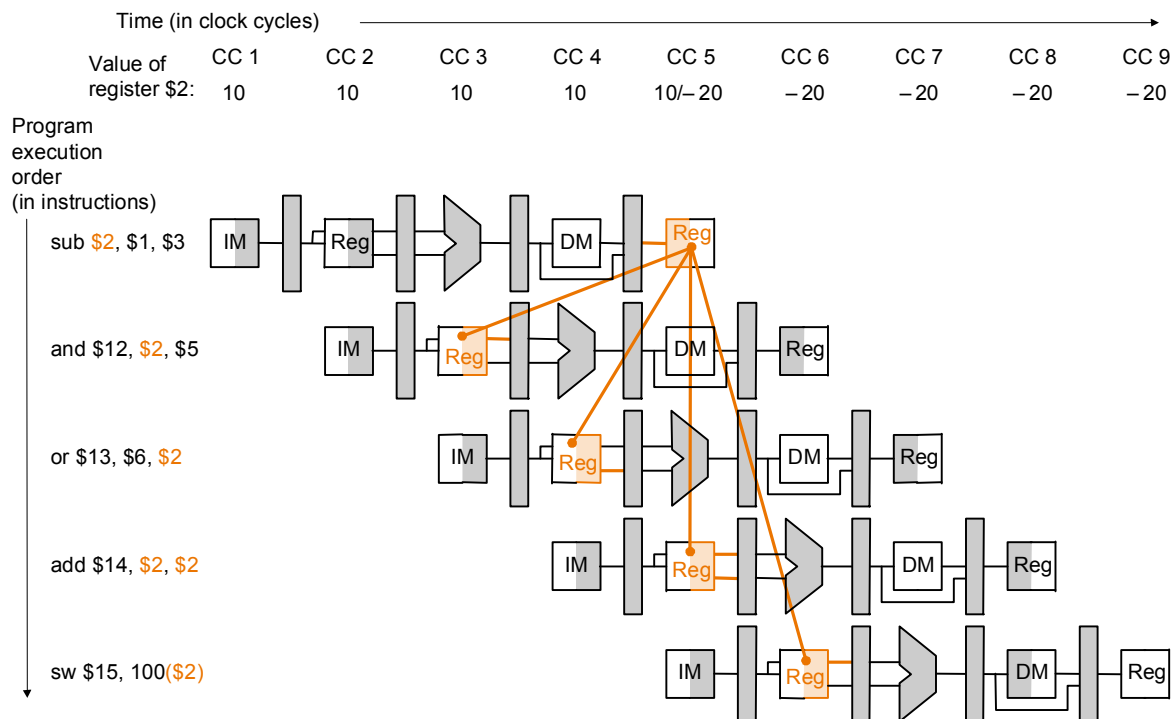


# Caminho de Dados com Pipeline



# Conflito de Dados

- A execução de uma instrução depende do resultado da outra
  - EX/MEM = ID/EX
  - MEM/WB = ID/EX



# Conflito de Dados

---

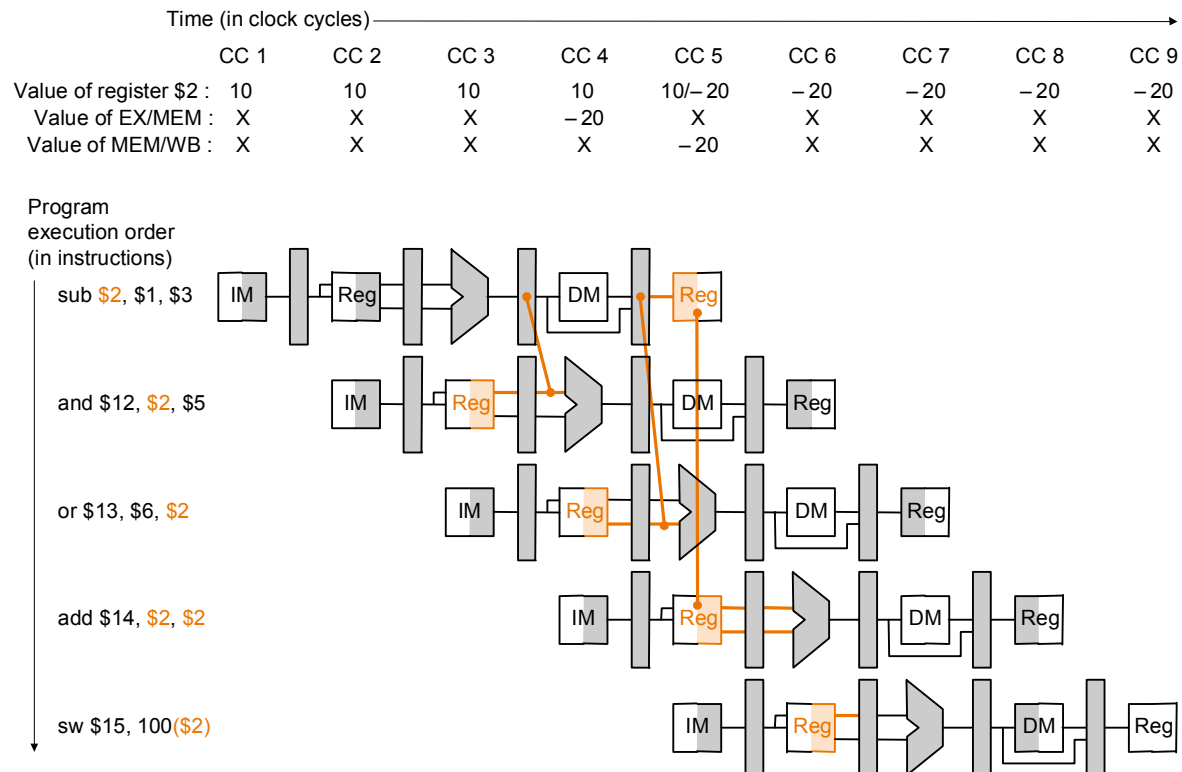
- **Solução por software**
  - O compilador garante que não há conflitos
  - Inserção de “nops” (operação vazia)
  - Onde inserir os “nops”?

```
sub    $2, $1, $3
nop
nop
and    $12, $2, $5
or     $13, $6, $2
add    $14, $2, $2
sw     $15, 100($2)
```

- **Problema: operações “nops” deixam a execução lenta!**

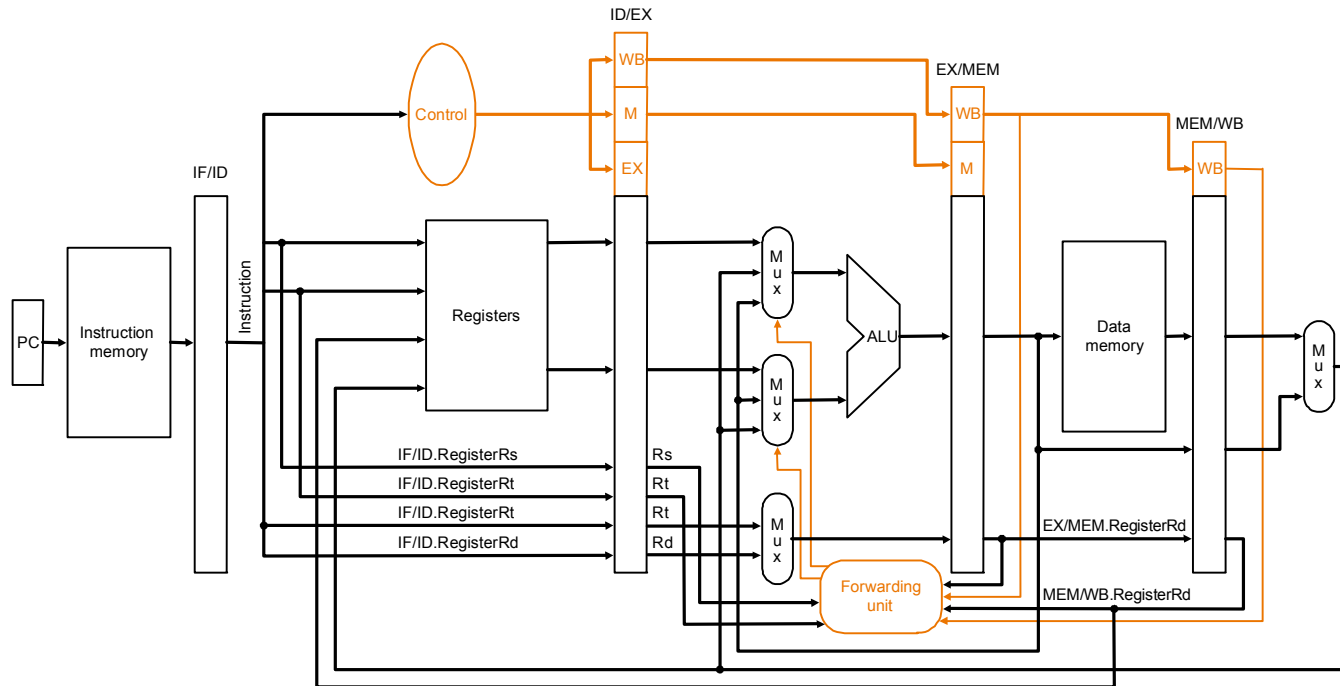
# Conflito de Dados

- **Adiantamento**
  - Utilizar resultados temporários (antes de serem escritos)
  - Adiantamentos do resultado da ULA
  - Adiantamento do dado a ser escrito na memória



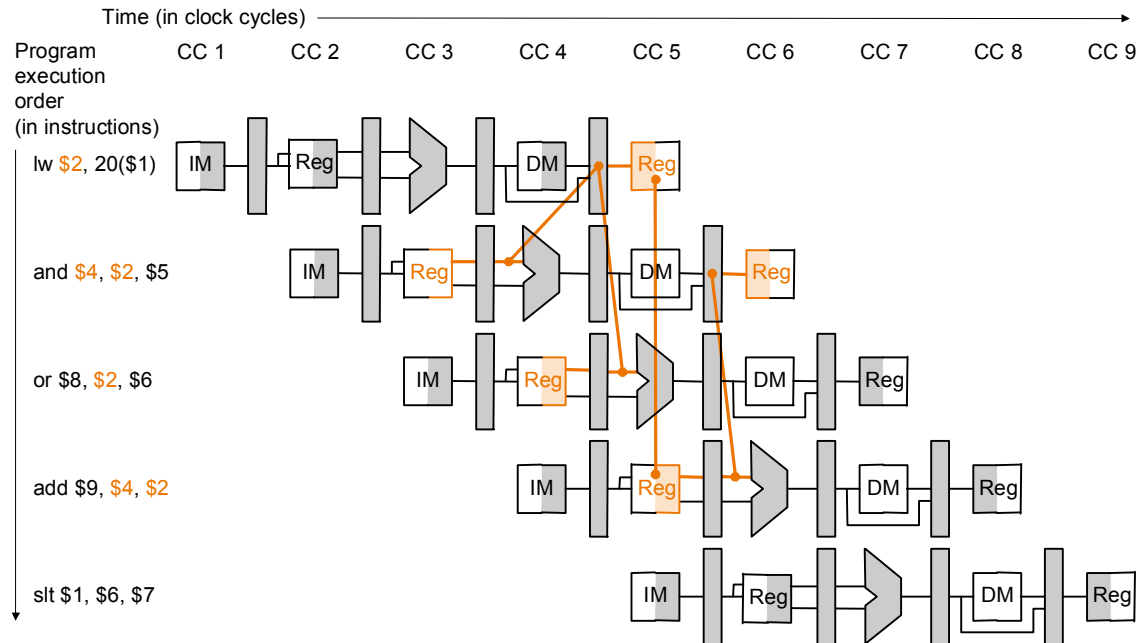
# Conflito de Dados

- Unidade de Adiantamento (Forwarding)



# Conflito de Dados

- Nem sempre é possível adiantar
  - Uma instrução tenta ler um registro que foi escrito por uma instrução lw
  - Valor só disponível em MEM/WB

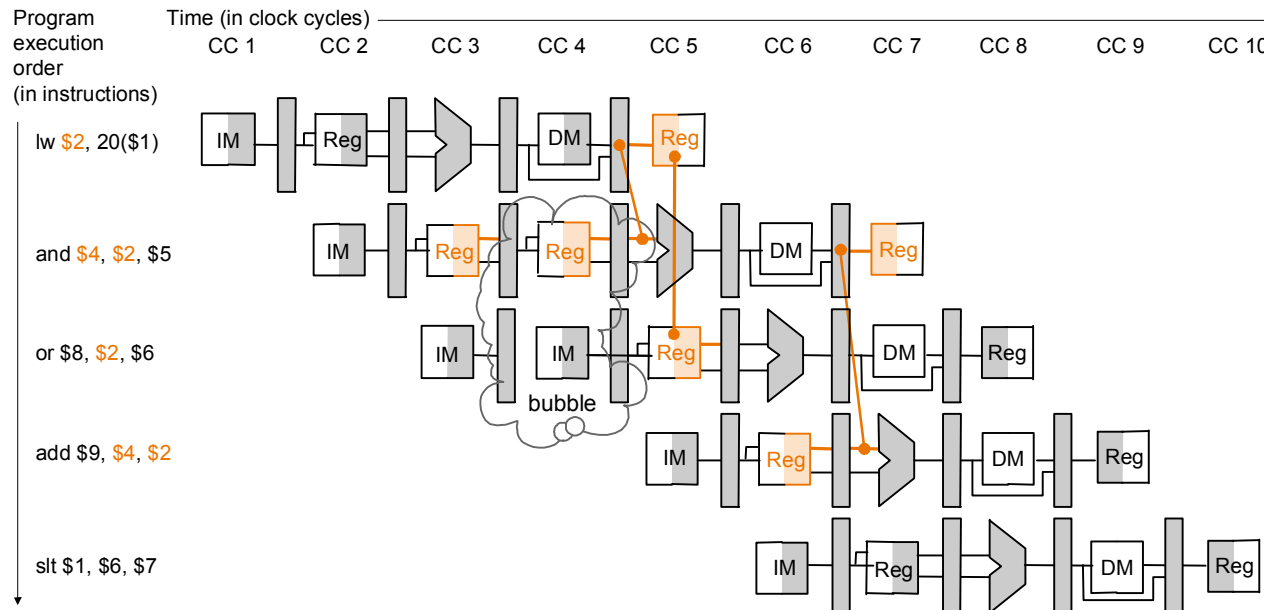


- Necessário uso de parada



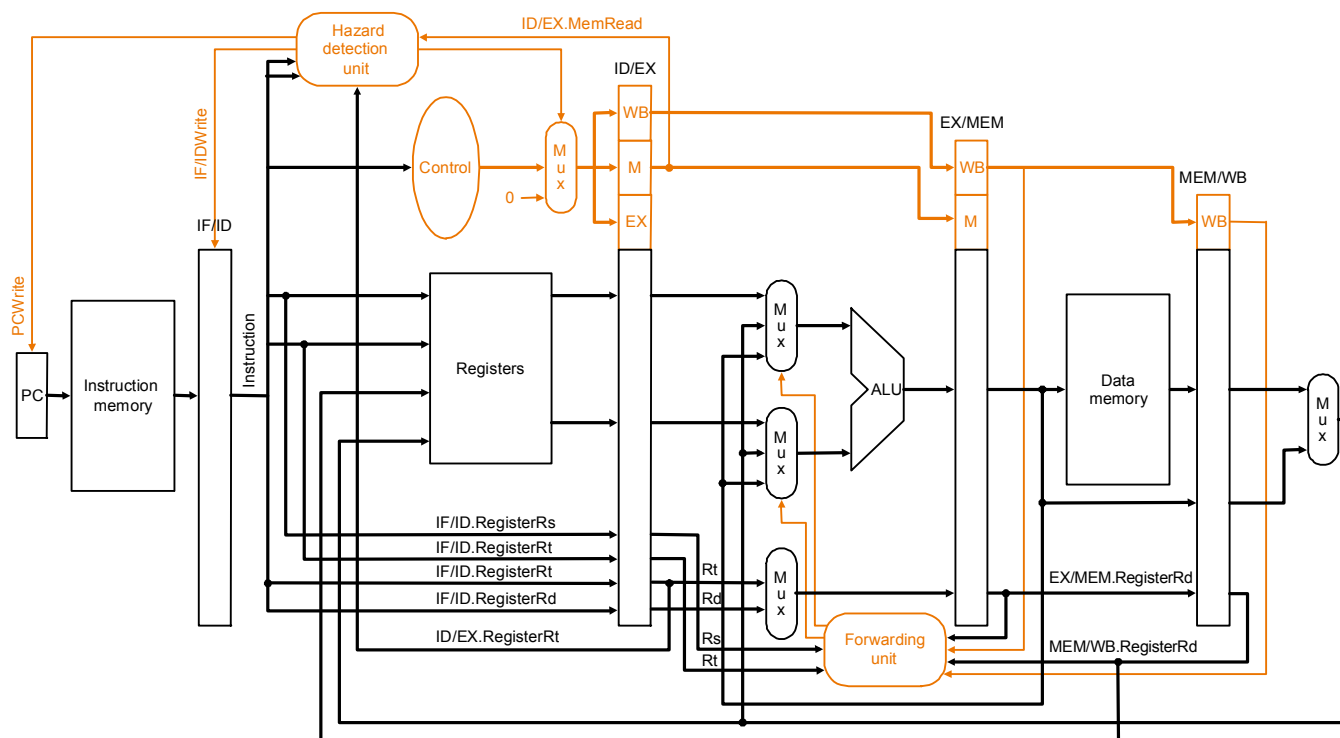
# Conflito de Dados

- **Paradas (stall)**
  - Para o pipeline mantendo as instruções no mesmo estágio
  - Diferente de utilizar “nop”, pois detecção de conflito ocorre durante o estágio
  - Criação de bolhas



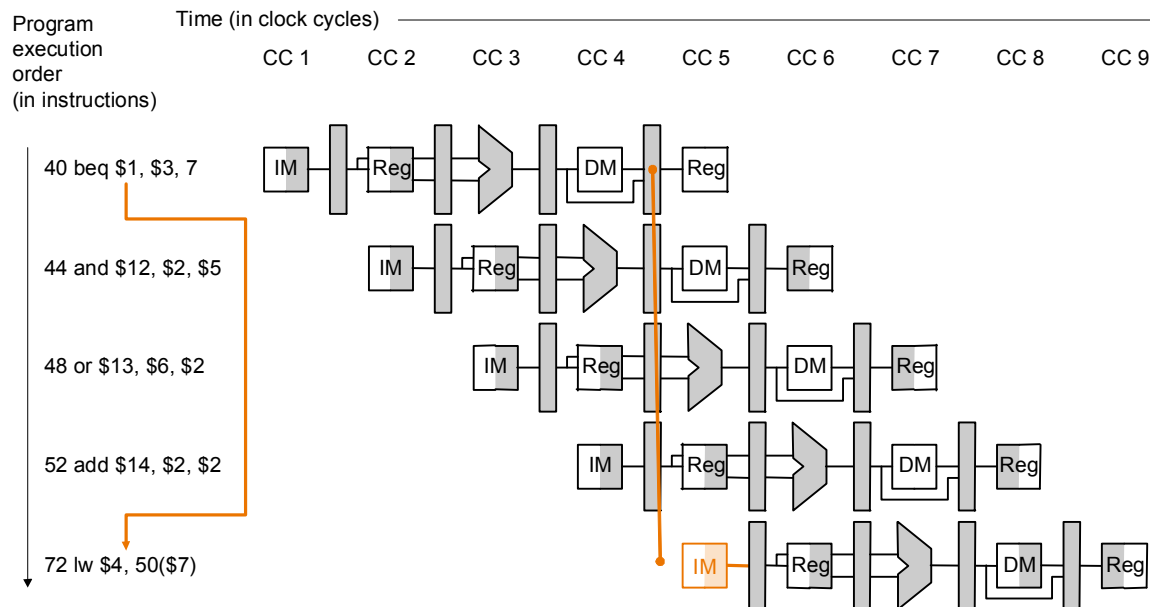
# Conflito de Dados

- **Unidade de Detecção de Conflitos (hazard)**
  - Parada é feita por meio de estágio que não escreve nada



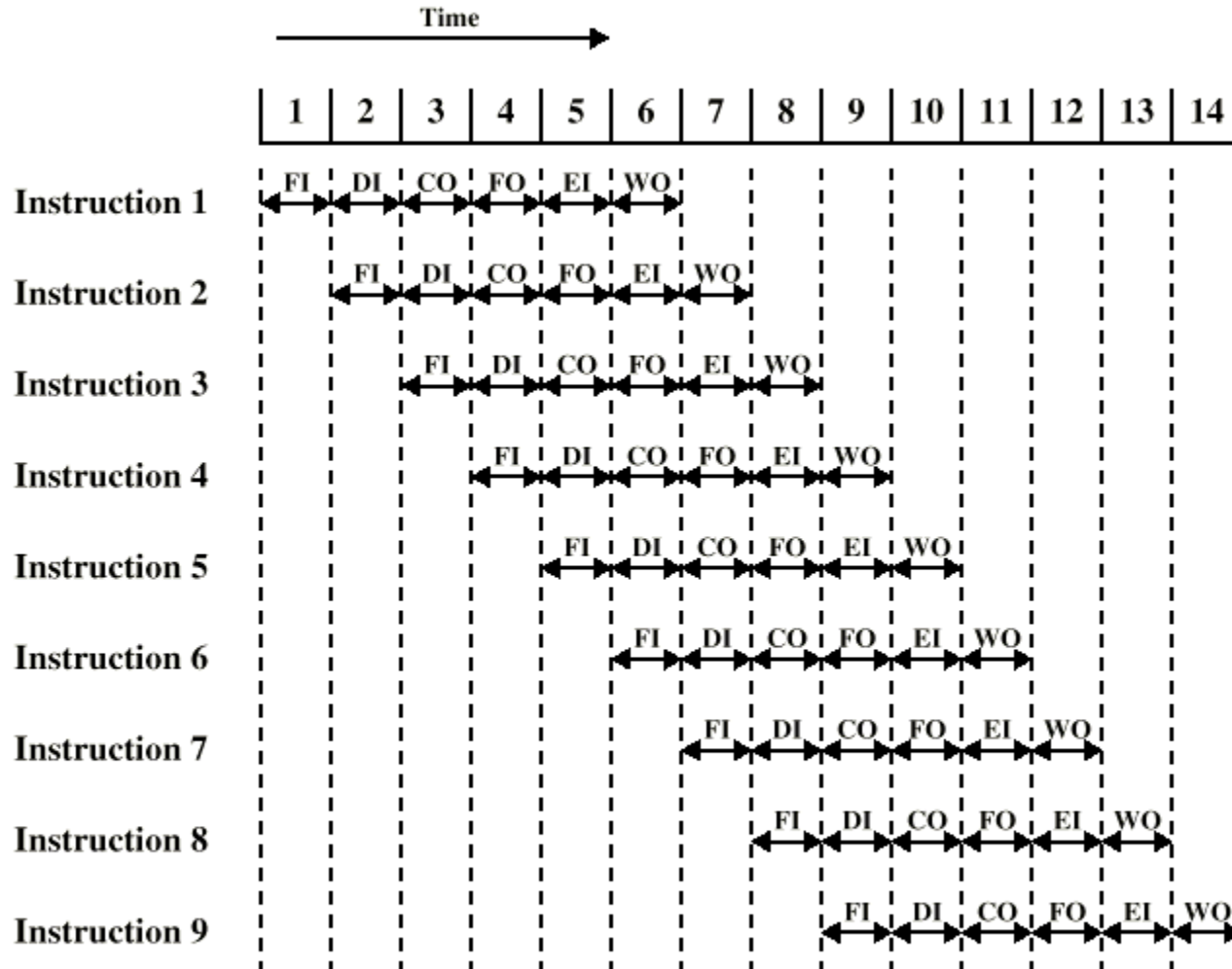
# Conflitos de Controle

- **Desvio condicional**
  - outras instruções estão no pipeline antes da decisão
  - **Predição: desvio condicional não-realizado**
  - **Necessário controle para eliminar instruções se desvio é realizado**



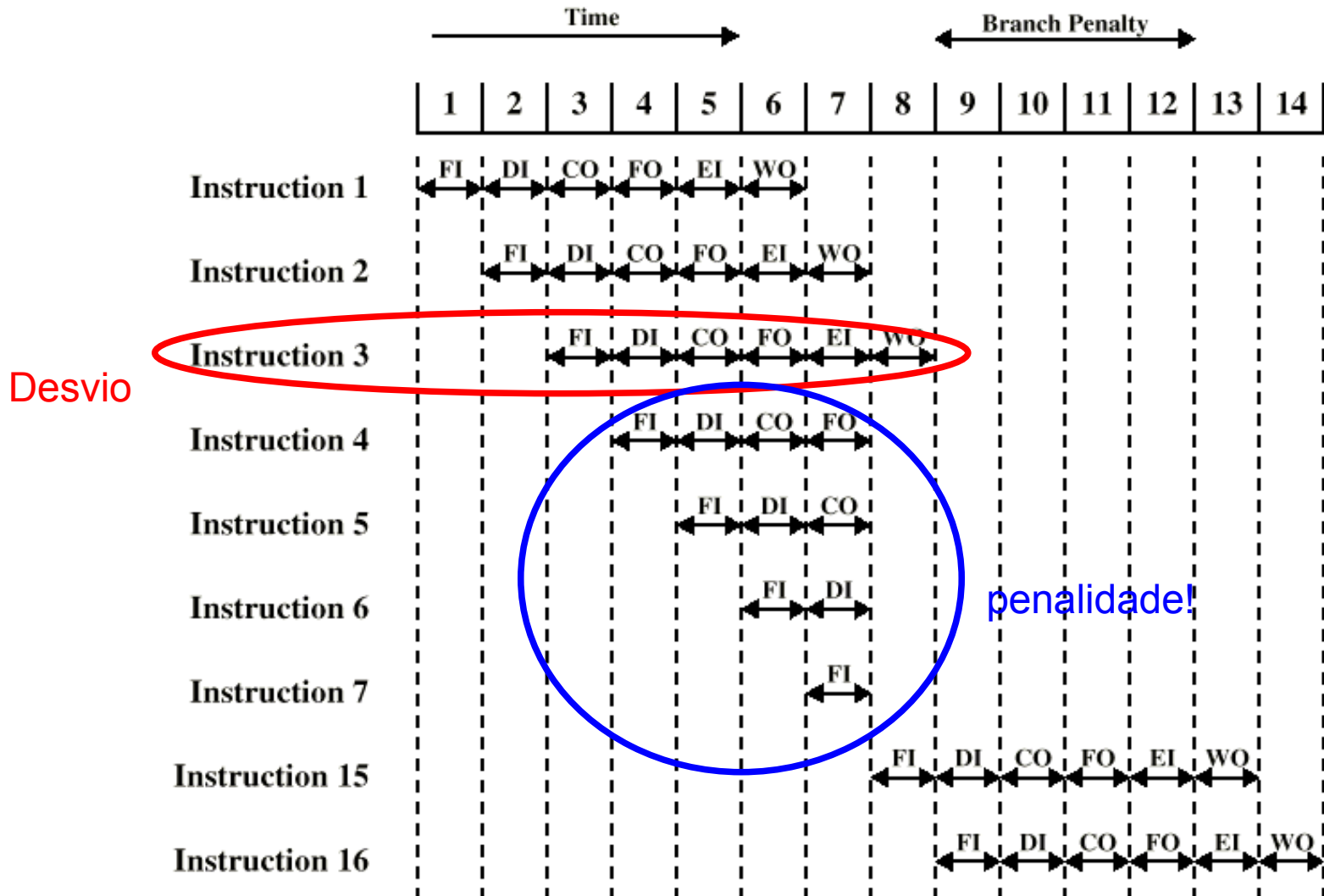
# Conflitos de Controle (pipeline Stallings)

- Predição Correta



# Conflitos de Controle (pipeline Stallings)

- Predição incorreta



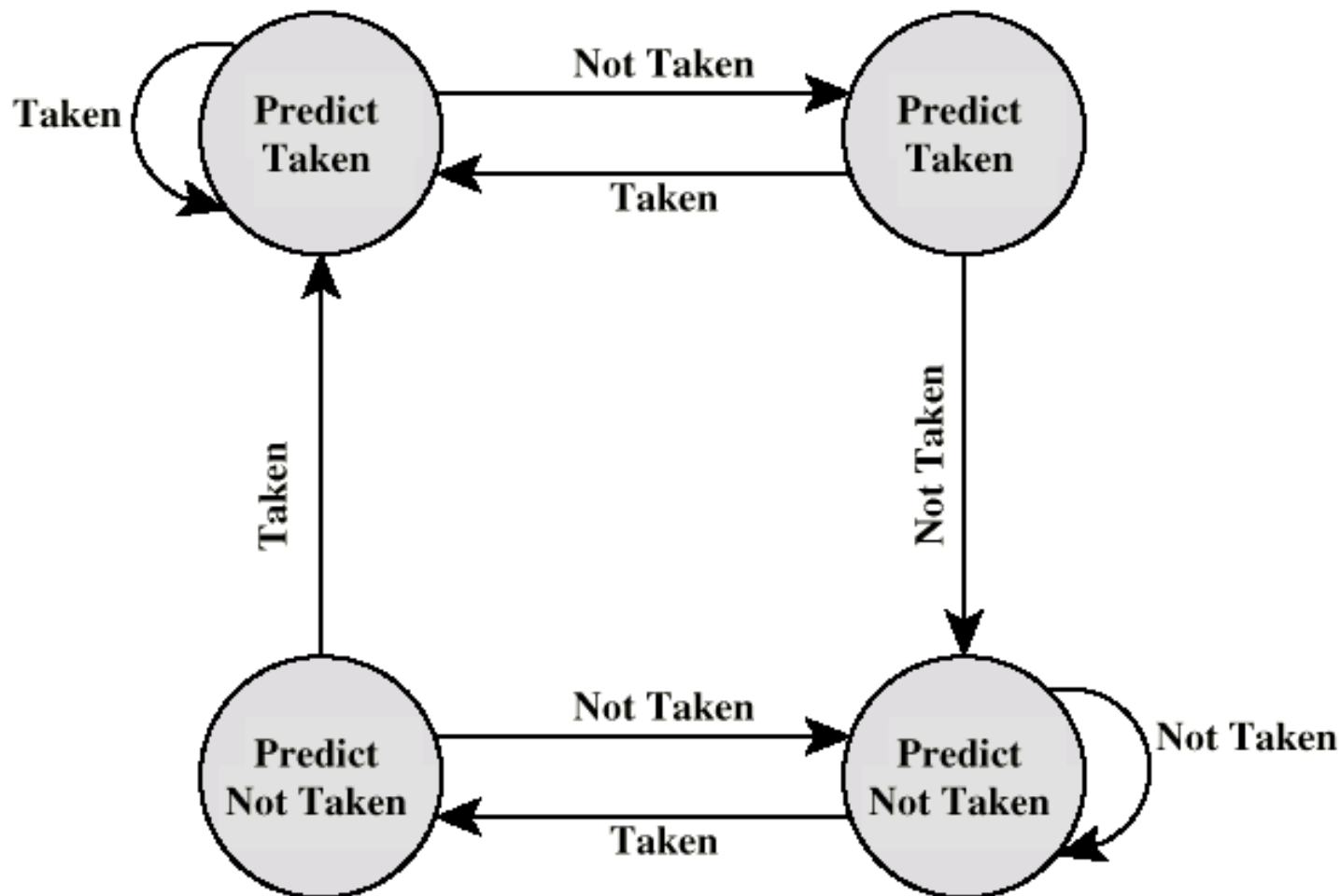
# Conflitos de Controle - Estratégias

---

- **Predição Desvio nunca realizado**
- **Predição Desvio sempre realizado**
- **Predição por Opcode**
  - Dependendo da instrução escolhe prediz desvio realizado ou não-realizado
- **Buffer para Predição (hardware especial)**
  - Mantém um histórico para cada instrução
  - Um ou Dois bits
  - Instrução mapeada pelos bits menos significativos (princípio da localidade)
- **Múltiplos Fluxos (hardware especial)**
  - Replica partes iniciais do pipeline
  - Após decisão, escolhe o pipeline correto
  - Problema: conflitos estruturais (memórias e registradores)
  - Problema: Instruções de desvios adicionais

# Conflitos de Controle

- Buffer para predição (Máquina de Estados)



# Melhorando o Desempenho

---

- Evitar paradas (reordenação das instruções)

|                               |                               |
|-------------------------------|-------------------------------|
| <code>lw \$t0, 0(\$t1)</code> | <code>lw \$t0, 0(\$t1)</code> |
| <code>lw \$t2, 4(\$t1)</code> | <code>lw \$t2, 4(\$t1)</code> |
| <code>sw \$t2, 0(\$t1)</code> | <code>sw \$t0, 4(\$t1)</code> |
| <code>sw \$t0, 4(\$t1)</code> | <code>sw \$t2, 0(\$t1)</code> |

- Evitar Penalidades (atraso do desvio)
  - Reordenação das instruções
  - A próxima instrução depois de um desvio é sempre executada
  - Confia no compilador para preencher o slot de atraso com algo útil
  - Não executa o desvio até que seja necessário



# Além do MIPS

---

- **Arquitetura MIPS**
  - Sem estágios interligados
  - Instruções com tamanho único
- **Outras arquiteturas**
  - Instruções com quantidade de ciclos diferentes (manter sequência da execução)
  - Estágios com tempo de ciclos diferentes (estágios interligados)
  - Instruções com tamanho diferentes (busca da próxima instrução)
  - Conflitos estruturais (espera por acesso aos estágios)
  - Conflitos de controle (predição de Desvios com Alteração de memória e registradores)
  - Conflitos de dados (leitura após escrita, escrita após leitura, escrita após escrita)

# Problema

---

**A) O slide 18 mostra uma solução para conflitos de dados implementada em duas unidades de controle: “Forwarding” e “Hazard Detection”. Esquematize uma solução para conflitos de controle (slides 20, 21 e 22).**

**B) A solução apresentada em sala considera o uso de duas memórias: instruções e dados. Considere uma organização que utilize apenas uma memória. Nessa organização pode ocorrer conflito estrutural. Em quais instruções esse conflito pode ocorrer? Esquematize uma solução para tais casos.**

**C) Qual é o throughput mínimo no cenário anterior?**